CodePilot AI — Presentation Guide

Use this as a 5–7 minute speaking guide with a live demo.

Opening — 20 seconds

CodePilot AI is a autonomous software engineering agent built to demonstrate both machine-learning reasoning and software-engineering delivery. The demo is local and reproducible, so I can explain every step without depending on a paid service.

Problem — 40 seconds

Developers spend time translating bug reports into code locations, preparing changes, and validating regressions. CodePilot demonstrates a controlled agent loop that automates the repetitive steps while keeping a human-review boundary.

Architecture — 60 seconds

Draw or explain the pipeline from input/data source to model/retrieval logic, then to the FastAPI service and dashboard. Emphasize the separation between model logic and product/API logic.

Live Demo — 2 to 3 minutes

Follow the numbered demo flow below. Pause at the algorithmic output and explain what the model/system is doing rather than clicking quickly through screens.

- Step 1: Open the dashboard and keep the pre-filled 100% coupon bug report.
- Step 2: Run analysis. Show that pricing.py is ranked first.
- Step 3: Show the one-line boundary-condition diff.
- Step 4: Compare failing tests before the patch with passing tests after it.
- Step 5: Explain that production deployment would add Git provider integration, containerized job isolation and policy checks.

Engineering — 45 seconds

Mention validation, tests, Docker, CI and the reason the demo can run locally. Then identify the optional components that would be used in a larger deployment.

Close — 30 seconds

Summarize one technical result, one design trade-off and one next improvement. Invite questions about model choice, metrics, scale or failure handling.

Resume-ready bullets

- Built an autonomous software-engineering agent that indexes a repository, retrieves bug-relevant files using vectorized code search, generates minimal patches, and validates changes with automated tests.
- Implemented a FastAPI workflow for bug triage, patch reasoning, unified-diff generation, sandboxed test execution, and human approval before code changes are accepted.
- Designed provider-agnostic LLM/RAG extension points, Dockerized deployment, API health checks, and CI testing for a production-style engineering workflow.

Metrics to mention if asked

- Retrieval recall@K
- Patch test-pass rate
- Regression rate
- Human acceptance rate
- Median time-to-validated-patch

30-second fallback if the live demo fails

> *“The application normally runs locally through FastAPI. I also keep a saved deterministic output and automated tests in the repository. I will show the expected output first, then walk through the code path that produces it and the test that verifies the behavior.”*

Rapid Interview Q&A;

Why sandbox execution?

Generated changes should not modify the source repository before validation and approval.

How would you scale it?

Persist embeddings in a vector index, incremental re-indexing on commits, isolate test jobs in containers, queue long-running tasks, cache repository metadata and stream status events.

What metrics matter?

File-retrieval recall@k, patch pass rate, regression rate, time-to-fix, false-positive patch rate and human acceptance rate.

Before the interview

- Run the app once from a clean terminal.
- Run `pytest -q` and confirm all tests pass.
- Know the main API endpoint and one representative request/response.
- Know one limitation and one scale-up plan.
- Do not claim optional technology as trained/deployed unless you have actually run it.